LLM Visualization

Conceptual Architecture for a Real-Time Transformer Microscope

Core idea	Build a scientifically faithful interactive environment where a user can follow a real prompt through a real open-source transformer - from text and tokens to embeddings, attention, residual/MLP transformations, logits, probabilities, and the next generated token. The visualization must expose real model data, not simulated animation.

1. What the project is

This project is an interactive LLM internal-state visualization and interpretability environment. It is not a chatbot interface and it is not only an attention heatmap. The system runs a real decoder-only language model, records its internal computations, and lets the user explore those computations at multiple levels of detail.

Central question: How can the internal processing of a transformer be made visible, navigable, and understandable without hiding the mathematics or overwhelming the user?

2. Design principles

Principle	Meaning in this project
Faithful	Every value shown originates from the real forward pass or a clearly defined derived transformation.
Hierarchical	Start simple, then drill down: model > layer > attention/MLP > head/projection > tensor/value.
Token-centered	A selected token remains identifiable while the user follows its representation through the network.
Real-time	Prompt changes produce a new trace; generation appends the chosen token and continues step by step.
Model-independent	The frontend uses a generic trace schema so Qwen, Llama, Gemma or future models can plug in.
No overclaiming	Expose observable internals without claiming that attention/activations alone reveal the model's true reasoning.

3. Starting model

Start with one small open-weight decoder-only model: **Qwen3-0.6B**. It is large enough to contain a realistic modern transformer stack while remaining practical for local tracing. The first implementation is a Qwen-specific adapter behind a model-independent interface.

Important	Do not build the frontend around Qwen-specific Python module names. Build a generic semantic trace - layer attention query/key/value, attention weights, residual state, MLP activation, logits - and let the Qwen adapter map model modules into those concepts.

4. Complete conceptual data flow

Stage	What happens	What the user should see
Input	User supplies text.	Prompt editor, run/generate controls, sequence length.
Tokenization	Text becomes model tokens and IDs.	Original text, token chips, token IDs, subword boundaries.
Embedding	Token IDs map to dense vectors; architecture handles position.	Vector summary, dimensions, optional 2D/3D projection.
Transformer blocks	Attention, residual paths, normalization and MLP update representations.	Horizontal layer chain with per-layer drill-down.
Final norm + LM head	Final state is normalized and projected to vocabulary logits.	Final state, top logits and vocabulary candidates.
Sampling + autoregression	Temperature/top-k/top-p shape the distribution; chosen token is appended.	Live probability changes, chosen token and generation timeline.

5. Anatomy of one transformer layer

The transformer layer must not appear as one opaque box. The project should let the user progressively open it and connect every displayed component to the real model computation.

Residual In > RMSNorm > Q/K/V > RoPE + Attention > O Projection > Residual Add > RMSNorm > MLP > Residual Add

5.1 Attention path

For a selected layer, expose the query, key and value projections produced from the current hidden state. Attention computes token-to-token scores and forms a weighted mixture of value vectors.

Core equation Attention(Q, K, V) = softmax(Q K^T / sqrt(d_k)) V. The interface should connect this equation to the selected layer/head values and the resulting attention matrix.

- Show token-by-token attention as a heatmap or connection view. Selecting a token such as "Germany" immediately emphasizes what that token attends to.
- Allow selection of layer and head. For grouped-query attention models, distinguish query heads from shared key/value heads where relevant.
- Expose shapes and concise numeric summaries first; show full vectors/matrices only on demand.

5.2 Residual stream

The residual stream is the continuous representation carried through the network. Attention and the MLP contribute updates that are added back into this stream.

Residual Before > + Attention Output > Residual Mid > + MLP Output > Residual After

This makes representation evolution visible: for the same token, compare the state before attention, after attention, and after the MLP within a layer.

5.3 MLP path

The feed-forward sublayer should also be inspectable. For a Qwen-style gated MLP, conceptually show normalization, gate/up projections, activation/gating, down projection, and the resulting residual contribution.

Hidden State > Norm > Gate + Up Proj. > Activation / Gating > Down Proj. > MLP Output

- Show which intermediate units are strongly active for the selected token.
- Show dimensions and distributions rather than dumping thousands of raw numbers by default.
- Preserve exact values for researcher-level inspection when requested.

6. Follow one token through the model

A persistent token selection should connect all visualizations. If the user selects a token, the interface keeps it highlighted while moving across layers and views. This turns separate charts into a coherent token journey.

Selected token stage	Question answered
Embedding	What representation does this token start with?
Layer 0 / 5 / 12 / 27	How does its hidden state change with depth?
Attention head	Which earlier tokens influence its attention update?
MLP	Which intermediate units activate for it?
Residual stream	How large is each layer update relative to the current state?
Final logits	Which vocabulary tokens become likely from the final representation?

7. Logits, probabilities and generation

After the final layer and normalization, the LM head produces a logit for each vocabulary token. Softmax converts logits to probabilities. Temperature changes distribution sharpness; top-k and top-p restrict the candidate set used for sampling.

Final Hidden State > LM Head > Raw Logits > Temperature > Top-k / Top-p > Probabilities > Chosen Token

For real-time generation, append the chosen token and repeat the process. Preserve each generation step so the user can inspect how internal states evolve as the sentence grows.

8. Visualization hierarchy and navigation

The main interface should be a horizontal, zoomable workspace. The user first sees the whole model and then opens only the component they care about. This prevents the screen from becoming unreadable.

Level	Default visual	Drill-down
Model overview	Input > Tokens > Embedding > L0 ... Ln > Logits > Output	Choose a layer or prediction step.
Layer view	Residual > Attention > Add > MLP > Add	Open attention or MLP internals.
Attention view	Q/K/V > heads > scores > softmax > output	Choose head/token; inspect heatmap and values.
MLP view	projection > activation/gating > projection	Choose token/unit; inspect activation distributions.
Token journey	Selected token across depth	Compare states, similarities, update magnitude and projections.
Output view	Top logits/probabilities	Adjust sampling controls and compare distributions.

9. Main visualization modes

Mode	Purpose
Architecture View	Navigate the complete horizontal model pipeline.
Layer Explorer	Inspect one transformer block in detail.
Attention Explorer	Compare heads, token relationships and attention patterns.
Token Journey	Follow a selected token through layer-by-layer representation changes.
Residual Stream View	Compare state before/after attention and MLP updates.
Representation Space	Project hidden states with PCA/UMAP/t-SNE for a semantic spatial view.
Logit Lens / Output View	Connect representations to vocabulary logits and likely next tokens.
Generation Timeline	Inspect the trace for each autoregressive generation step.
3D City of Meaning	Use genuine hidden-state geometry as an advanced spatial view, not a separate fake simulation.

10. Technical system architecture

Next.js UI > **Trace API** > **Model Adapter** > **PyTorch / HF Model** > **Trace Collector** > **Trace Store**

The backend owns all model computation and tracing. The frontend never reimplements transformer mathematics or invents values. A model adapter translates architecture-specific modules into a shared trace schema. The collector records tensors and summaries during inference; the trace store keeps active traces so the frontend requests only the detail currently being inspected.

Component	Responsibility
Frontend - Next.js / React / TypeScript	Horizontal explorer, selections, charts, 2D/3D views, generation controls.
FastAPI backend	Run traces, serve metadata/trace slices, manage generation steps.
ModelAdapter	Map Qwen-specific modules into generic transformer concepts.
TraceCollector	Capture hidden states, attention, projections, residual/MLP outputs and logits.
TraceStore	Keep active trace data indexed by trace ID; move to disk/cache later if needed.
Analysis utilities	PCA/UMAP, similarities, norms and other clearly labeled derived measurements.

11. Trace API concept

Avoid returning every tensor in one enormous JSON response. The first request returns an overview and a trace ID. Detailed data is fetched only when a user opens a layer, head, token, or generation step.

Conceptual endpoint	Returns
POST /trace	Run prompt; trace ID, tokens, model metadata, layer list, next-token summary.
GET /trace/{id}/layers	Lightweight per-layer summaries and tensor shapes.
GET /trace/{id}/layer/{L}	Residual/attention/MLP summary for one layer.
GET /trace/{id}/layer/{L}/attention	Heads, attention matrices and Q/K/V summaries.
GET /trace/{id}/layer/{L}/mlp	MLP projection/activation summaries.
GET /trace/{id}/token/{T}/journey	Selected-token representations and updates across layers.
GET /trace/{id}/logits	Raw/top logits, probabilities and sampling information.

12. Generic trace schema

The frontend should reason in semantic concepts instead of framework module paths. A trace identifies the model, prompt, tokens, generation step and a list of layer records. Each layer record exposes standardized fields where supported: input state, normalized state, Q/K/V, attention weights/output, residual-after-attention, MLP intermediates/output and residual-after-MLP. Unsupported values are marked unavailable rather than fabricated.

Schema rule	Model-specific details belong inside adapters. The UI asks for concepts such as layer[4].attention.query or layer[4].residual_after_mlp, not qwen.layers.4.self_attn.q_proj.

13. Build order - foundation to full research system

Phase	What must be true before moving on
1. Model baseline	Qwen loads locally; tokenization and next-token logits are correct.
2. Trace baseline	Hidden states and attention outputs can be captured reliably.
3. Deep instrumentation	Q/K/V, residual updates, MLP internals and final logits are captured or transparently derived.
4. Generic schema	Trace format is independent of Qwen-specific Python names.
5. API	Frontend can request overview and lazy-loaded trace slices by trace ID.
6. Horizontal explorer	Input-to-output model pipeline works and layers are clickable.
7. Internal drill-down	Layer/attention/MLP views expose real values and shapes.
8. Token journey	One token stays selected and can be followed across depth.
9. Generation	Autoregressive steps are traceable; logits/sampling controls use real output.
10. Advanced views	Representation space, logit-lens-style analysis, 3D City of Meaning, comparisons and more models.

14. What the professor should be able to test

A strong research demonstration should survive technical questioning. The following checks should be easy to perform:

- Enter a new prompt and demonstrate that all displayed values change because a new real forward pass was performed.
- Choose a token, layer and attention head and show the corresponding real attention pattern and tensor dimensions.
- Explain one layer mathematically while pointing to matching visual components and actual model values.
- Show the residual representation before and after attention/MLP updates, proving the visualization follows the transformer computation rather than isolated charts.
- Show final logits and how softmax/sampling settings affect the visible next-token distribution.
- Explain what is directly observed from the model versus what is derived analysis such as PCA, similarity, norms or clustering.

15. Scientific boundaries

Do not claim	"This attention head proves why the model thought X." Attention, activations and hidden states are observable internal signals, but they are not automatically a complete causal explanation of model reasoning.

A defensible description is: the project visualizes and analyzes internal transformer states and transformations so researchers and learners can inspect how representations, token interactions and output distributions evolve during inference.

16. Definition of success

Successful project	Not enough
A fresh prompt runs through the actual model and produces a trace.	A fixed animation that always follows the same path.
A selected token can be traced across layers with real hidden-state data.	Only showing tokens and the final response.
Layers open into attention, residual and MLP internals.	One opaque "Transformer" box.
Attention heads and matrices match the chosen layer/head/token.	Decorative connection lines not tied to tensors.
Logits and probability controls use real model output.	Hard-coded probability bars.
Architecture supports additional model adapters later.	Frontend tightly bound to one Qwen module path.
Advanced 3D views are alternate views of the same trace.	A beautiful 3D scene disconnected from real inference data.

17. Full conceptual architecture at a glance

The complete project can be understood as one continuous system with three layers: **model execution**, **trace/analysis infrastructure**, and **interactive visualization**. Keeping these layers separate is what makes the system accurate, extensible, and reusable.

Layer	Core responsibility	Examples
Model execution	Run the real open-source LLM and expose architecture-specific internals.	Tokenizer, embeddings, attention, MLP, residuals, final logits.
Trace infrastructure	Capture, normalize, store and serve internal states through a generic schema.	Hooks/model outputs, trace IDs, lazy loading, token journey, layer summaries.
Interactive visualization	Turn the trace into navigable views without changing the underlying data.	Horizontal architecture, heatmaps, residual view, PCA/UMAP, 3D City of Meaning.

18. Future multi-model extension

Once Qwen is stable, add new adapters rather than rewriting the interface. Each adapter answers the same semantic questions: how to load the model, where the transformer layers live, how to access attention/MLP components, how to expose hidden states and logits, and which concepts are unsupported. The UI can then offer model selection while preserving the same interaction model.

Generic capability	Qwen adapter now	Future adapter requirement
Tokenize / detokenize	Implemented with Qwen tokenizer.	Map equivalent tokenizer behavior.
Layer enumeration	Map Qwen transformer blocks.	Expose ordered decoder blocks.
Attention trace	Map Q/K/V, head outputs, weights where available.	Provide equivalent attention concepts or mark unsupported.
MLP trace	Map gated MLP projections/activation/output.	Expose architecture-specific MLP through common fields.
Residual trace	Record pre/post attention and MLP states.	Expose equivalent residual checkpoints.
Output trace	Expose final norm, LM head, logits and sampling.	Return standard vocabulary-output representation.

19. One-sentence project description

Project statement	An interactive real-time visualization environment that lets users trace how an open-source transformer processes text - from tokenization and embeddings, through attention, residual streams and MLP transformations across its layers, to vocabulary logits, sampling and the next generated token.

20. Practical implementation baseline

Area	Initial choice	Reason
Model	Qwen3-0.6B	Small enough for local tracing while still representing a modern decoder-only transformer.
Inference	PyTorch + Hugging Face Transformers	Direct access to model modules, outputs and hooks.
Backend	FastAPI	Clear Python boundary around model execution and trace endpoints.
Frontend	Next.js + React + TypeScript	Strong interactive UI foundation for horizontal exploration and drill-down.
2D visualization	SVG/React components + Plotly where useful	Precise architecture diagrams plus heatmaps/distributions.
3D visualization	Three.js / React Three Fiber later	Advanced semantic-space exploration on top of the same trace.

21. Conceptual checklist before calling the system "real"

- No hard-coded attention patterns, hidden states, logits or token probabilities.
- Every component view names its tensor shape and whether the value is raw model output or derived analysis.
- The user can move from a high-level layer to the exact internal data without losing the selected prompt/token context.
- The same stored trace can power 2D architecture, attention, token journey, residual, logit and 3D views.
- Changing the prompt creates a genuinely new trace; changing sampling settings operates on the real output distribution.
- Unsupported model internals are shown as unavailable, never approximated silently.

22. Official references for implementation

Recommended primary references: the Qwen3-0.6B model card and Qwen3 documentation in Hugging Face Transformers; Hugging Face causal-language-model output documentation; and PyTorch module forward-hook documentation. These should be treated as implementation references, while this document remains the conceptual blueprint.

Bottom line	Run the real model -> capture the internals -> normalize them into a generic trace -> visualize the trace hierarchically -> preserve a selected token across views -> connect final hidden states to real logits and generation. Everything else, including 3D, should be an additional view of the same scientific data.